MuJoCo Drone Simulation

Implementation Summary Report

17 Python Modules | Physics-Based PID Control | Subsumption Architecture | A* / Dijkstra Pathfinding
Frontier Exploration | IMU Trajectory Replay | Multi-Sensor Fusion | Post-Mission Analytics

Document Type	Technical Implementation Summary
Project	Autonomous Drone Simulation — MuJoCo
Date Generated	June 25, 2026
Modules Covered	17 Python source files
Output Files	13 runtime-generated data files

Table of Contents

■ 1. Architecture Overview

The project is a fully autonomous, physics-based drone simulation built on MuJoCo. It is structured in decoupled, modular layers where each module has a single responsibility. The main orchestrator simulate.py wires all components together and drives the real-time simulation loop.

Layer	Module(s)	Role
Orchestrator	simulate.py	Main loop, state machine, A*/Dijkstra
Physics	DronePhysicsController	Force-based PID altitude + attitude control
Autopilot	DroneAutopilot	High-level command dispatch + sensor wiring
Behavior Control	subsumption.py (6 behaviors)	Priority-based reactive behavior suppression
Sensing	sensor_simulation.py	GPS, IMU, LiDAR, barometer, magnetometer
Fusion	sensor_fusion.py	Weighted multi-sensor pose estimation
Mapping	occupancy_grid_mapping.py	Probabilistic SLAM-style occupancy grid
Terrain Discovery	terrain_scanner.py	Fog-of-war reveal with directional FOV
Exploration	frontier_detection.py + ranking.py	Boundary detection + ranked target selection
Coverage Analytics	coverage_metrics.py	Exploration % tracking + auto-return trigger
Path Return	imu_trajectory.py	Record path outbound, replay reversed to home
Mission Control	mission_manager.py	High-level mission state lifecycle
Post Processing	post_mission_analysis.py	Offline inter-pad route cost matrix
Obstacles	dynamic_obstacles.py	Moving obstacle grid updates + replan trigger

■ 2. Core Flight System

■ 2.1 DronePhysicsController (simulate.py)

Implements purely MuJoCo-based force injection via data.xfrc_applied. No qpos manipulation is used — the drone is controlled exclusively through applied forces and torques. The controller runs at every physics step (200 Hz default).

Control Axis	Type	Gains	Notes
Altitude (Z)	PID	KP=28.0 KI=4.5 KD=12.0	Integral clamped ±8.0 to prevent windup
Horizontal XY	PD	KP=18.0 KD=9.0	Max lateral force 30 N; mass-scaled
Roll / Pitch	PD	KP_ATT=6.0 KD_ATT=2.5	Quaternion → RPY via closed-form formula
Yaw	P	KP_YAW=3.0	Shortest-angle error; wraps ±π
Linear Drag	Damp	K_DRAG_LIN=1.8	Applied to all 3 velocity axes
Rotational Drag	Damp	K_DRAG_ROT=0.6	Reduces oscillation on attitude axes

■ 2.2 A* Pathfinder (simulate.py → run_astar)

8-connected grid search using the **octile distance heuristic** (admissible for diagonal moves). Cells adjacent to occupied cells receive a +3.0 inflation cost penalty to keep the drone away from wall edges. The planner can be switched to Dijkstra at runtime via the A key.

Feature	Implementation Detail
Heuristic	Octile: max(dx,dy) + (√2–1)·min(dx,dy) — admissible, consistent
Connectivity	8-directional (cardinal 1.0, diagonal 1.414)
Obstacle Inflation	+3.0 extra cost for cells within 1-cell of any obstacle
Data Structure	Python heapq min-heap on f_score
Fallback	Dijkstra (run_dijkstra) — selectable at runtime
Grid Resolution	160×160 cells over 16×16 m world (0.1 m/cell)

■ 2.3 DroneAutopilot State Machine (simulate.py)

State	Entry Condition	Exit Condition
LANDED	Power-on / touchdown complete	L key pressed → TAKEOFF
TAKEOFF	L key from LANDED	Altitude within 10 cm of hover_height
HOVER	Takeoff complete / scan done	C key (SCANNING), cmd_goto (GOTO_TARGET)
SCANNING	C key while HOVER	360° yaw accumulated (2π rad)
GOTO_TARGET	S key + target pad set	Within 30 cm of target pad center

State	Entry Condition	Exit Condition
AUTOLAND	Arrival at target pad	Touchdown + 2 s wait → LANDED
LANDING_HOME	L key while flying / kill command	Terrain height reached → LANDED

■ 3. Subsumption Architecture — Behavior Stack

Implements Brooks' **Subsumption Architecture** — a reactive, layered control system where higher-priority behaviors suppress lower-priority ones. The BehaviorManager.step() method iterates the sorted behavior list each simulation tick and executes the first behavior whose check_trigger() returns True, skipping all lower-priority behaviors.

Priority	Behavior Class	Trigger Condition	Action Taken
5 ■	KillSwitchBehavior	kill_switch_active == True	Zero all forces, state→LANDED, clear path
4 ■	EmergencyLandingBehavior	state == 'LANDING_HOME'	PID descent in-place to terrain height
3 ■	ObstacleAvoidanceBehavior	state=='GOTO_TARGET' AND path_blocked	Re-run A*/Dijkstra, rebuild waypoints
2.5 ■	ReturnHomeBehavior	auto_return_active == True	Pull reversed trajectory waypoints, steer home
2 ■	NavigationBehavior	state in (TAKEOFF, HOVER, GOTO_TARGET, AUTO_LAND)	Normal takeoff/hover/path-follow/autoland
1 ■	TerrainScanningBehavior	state == 'SCANNING'	Apply spin torque; end at 2π accumulated yaw

■ 3.1 ReturnHomeBehavior — Trajectory Reversal Detail

When auto_return_active is set (triggered at 70% coverage or zero remaining frontiers), this behavior takes over from NavigationBehavior. It queries TrajectoryReplayer.get_next_waypoint(dx, dy) each tick and issues velocity-mode physics commands toward each reversed waypoint. On completion (home radius reached), it sets state→LANDING_HOME which hands off to EmergencyLandingBehavior (priority 4) for final descent.

4. Mapping & Exploration Pipeline

4.1 TerrainScanner (terrain_scanner.py)

Implements a **fog-of-war** reveal system. The drone sees only what its sensors have scanned. Two grids are maintained in parallel:

Grid / Mask	Type	Purpose
discovered_grid	int8 array	Holds obstacle status for explored cells only
explored_mask	bool array	True if the cell has been seen by the sensor

Each step: drone grid position is computed, cells within sensor_radius=2.5 grid units are checked against a fov_deg=90° forward cone. Only cells inside the cone are marked as explored. The planning grid returned to A* always uses only explored cells.

4.2 OccupancyGridMapping (occupancy_grid_mapping.py)

Probabilistic occupancy mapping using a **log-odds update rule**. Each sensor reading increments or decrements the log-odds for a cell. The grid stores three cell states:

Value	Constant	Meaning
-1	CELL_UNKNOWN	Never observed by any sensor
0	CELL_FREE	Observed and confirmed clear
1	CELL_OCCUPIED	Obstacle detected

4.3 FrontierDetection (frontier_detection.py)

Identifies the boundary between CELL_FREE and CELL_UNKNOWN space using a BFS/flood-fill over the occupancy grid. Contiguous frontier cells are grouped into labeled **regions**, each with a centroid (world + grid coordinates) and a size (cell count). Regions below a minimum size threshold are discarded as noise.

4.4 CoverageMetrics (coverage_metrics.py)

Computes exploration progress after every mapping update. Coverage percentage = (free_cells + occupied_cells) / total_cells × 100. Provides get_statistics() returning a dictionary with area covered, cell counts by type, elapsed mission time, and current coverage %. When coverage ≥ 70%, calls autopilot._trigger_auto_return().

5. Sensor Suite & Fusion

5.1 SensorSimulationSuite (sensor_simulation.py)

Emulates all sensors a real autonomous drone would carry. Each sensor adds configurable Gaussian noise via NumPy to simulate realistic measurement error.

Sensor	Output	Noise Model
GPS	x, y, z position	Gaussian σ=0.05 m horizontal, 0.1 m vertical
IMU	acceleration (ax,ay,az), gyro	White noise + bias drift
Barometer	altitude (z)	Gaussian σ=0.02 m
LiDAR	range distance per ray	Random ray miss probability + σ=0.01 m
Magnetometer	heading angle	Gaussian σ=0.5°

5.2 SensorFusion (sensor_fusion.py)

Combines GPS and IMU into a single FusedState dataclass (x, y, z, vx, vy, vz, valid). GPS provides position ground truth; IMU velocity integration fills time gaps between GPS fixes. Weighted average fusion: GPS weight 0.8, IMU weight 0.2 when both valid. Output is consumed by **FrontierRanking** and **CoverageMetrics**.

■ 6. Mission Management & Post-Mission Analysis

■ 6.1 MissionManager + MissionState (mission_manager.py / mission_state.py)

MissionState is a Python Enum defining five lifecycle stages. MissionManager validates state transitions against an allowed adjacency table, preventing illegal jumps (e.g., IDLE→LAND). A force_state() method is provided for emergency overrides.

State	Entered When	Exits To
IDLE	Simulation starts	EXPLORE
EXPLORE	E key pressed while hovering	RETURN_HOME
RETURN_HOME	Coverage ≥ 70% or no frontiers remain	LAND
LAND	Trajectory reversal complete, home reached	MISSION_COMPLETE
MISSION_COMPLETE	Drone has landed at home pad	— (terminal)

■ 6.2 PostMissionAnalysis (post_mission_analysis.py)

Offline analysis executed automatically after the drone lands. Runs both A* and Dijkstra between **every pair** of the 6 landing pads on the final discovered occupancy grid. Produces:

- A route cost matrix (JSON) — inter-pad travel costs for both algorithms
- mission_route_map.png — All inter-pad paths rendered on the explored map
- Console-printed route matrix via print_route_matrix()

■ 7. Frontier Detection & Ranking

■ 7.1 FrontierRanking (frontier_ranking.py)

Scores every detected frontier region using a **four-component weighted formula**. The best-scored frontier becomes the drone's next autonomous exploration target.

Component	Weight	Computation Method	Normalization
Information Gain	40%	Count unique CELL_UNKNOWN neighbors in 8-direction	min(1.0, count / 50)
Travel Cost	30%	Run A* from drone to frontier centroid; apply exponential decay	exp(−0.05 × A* cost)
Safety Score	20%	Window search (6-cell radius) for nearest obstacle	min(1.0, dist / 6.0)
Reachability	10%	Binary: 1.0 if A* finds a path, 0.0 if unreachable (skip entirely)	2 of 4

Caching optimization: The ranker stores a cache key of (pos_key, grid_sum, regions_len). If all three match the previous call, the full re-evaluation is skipped, avoiding redundant A* calls at every simulation tick.

Tie-breaking: Final sort uses key=lambda x: (score, –region_id) in descending order, ensuring deterministic selection when scores are equal.

■ 8. Trajectory Recording & Auto-Return

■ 8.1 TrajectoryRecorder (imu_trajectory.py)

Records the drone's flight path during the exploration phase. Sampling is rate-limited by two filters applied before storing a waypoint:

Filter	Value	Purpose
Temporal sampling	10 Hz	record_hz=10.0 — at most 10 samples per second
Distance filter	0.05 m	min_dist_m=0.05 — ignores stationary hovering

Provides get_log() returning the full waypoint list and get_total_distance_m() for mission statistics.

■ 8.2 TrajectoryReplayer (imu_trajectory.py)

Receives the recorded log, **reverses the waypoint list**, and serves waypoints one at a time. The drone navigates toward each reversed waypoint using the same velocity-mode physics commands as normal path following. Home arrival is detected when the drone enters a home_radius_m=0.40 m circle around the home coordinates.

■ 8.3 Auto-Return Trigger Logic

The auto-return sequence is initiated by DroneAutopilot._trigger_auto_return() when either condition is met:

- Coverage ≥ 70% (configurable via explore_coverage_threshold)
- No reachable frontier regions remain after FrontierRanking update

The method stops the recorder, transitions MissionState → RETURN_HOME, starts the replayer, and sets auto_return_active = True which activates ReturnHomeBehavior (priority 2.5) in the subsumption stack.

■ 9. Utilities, Assets & Report Generation

File	Purpose	Method
add_rocks.py	Inject rock obstacles into scene.xml	Random pos (pad-zone excluded) → <geom> XML entries
generate_assets.py	Pre-generate terrain heightmap + grid PNG	PIL/NumPy from get_terrain_height() math formula
generate_pdf.py	Generate project_report.pdf	ReportLab A4 multi-section technical report
generate_manual_pdf.py	Generate drone_operations_manual.pdf	ReportLab formatted operations manual
run.bat	One-click simulation launcher	Batch: activate env → generate_assets.py → simulate.py
dynamic_obstacles.py	Moving obstacle tracking + replan trigger	MuJoCo body positions → grid coords → path_blocked flag

■ 10. Runtime Output Files

The simulation automatically generates the following data files at runtime. No manual export is required — all files are written to the project directory.

File	Generated By	Contents
flight_trajectory.csv	simulate.py	Per-step: time, state, x/y/z, quaternion, velocity, forces
detection_log.csv	simulate.py	Event log: takeoff, hover, landing, scan events
ground_map.png	simulate.py	Fog-of-war explored map rendered as PNG
shortest_path_map.png	simulate.py	Map with A* path overlay drawn on top
map_data.json	simulate.py	Full grid array, pad locations, path coordinates
occupancy_grid.csv	occupancy_grid_mapping.py	Raw grid cell values (–1 / 0 / 1)
map_metadata.csv	simulate.py	Grid resolution, origin, world dimensions
landing_spots.csv	simulate.py	All pad names, world + grid coordinates
visiting_sequence.csv	simulate.py	Pad visit order during mission
sensor_metrics.json	sensor_simulation.py	Sensor noise statistics and accuracy metrics
mission_log.json	simulate.py	Coverage stats, trajectory snapshot count, end time
mission_route_analysis.json	post_mission_analysis.py	Inter-pad A*/Dijkstra cost matrix for all pad pairs
mission_route_map.png	post_mission_analysis.py	Map with all inter-pad routes visualized

■ 11. Keyboard Controls Reference

All simulation controls are handled in the MuJoCo viewer key callback. Controls operate on the drone's state machine and trigger the appropriate autopilot command methods.

Key	Action	Method Called	Precondition
L	Takeoff	cmd_takeoff()	state == LANDED
L	Return Home + Land	cmd_takeoff()	state == HOVER/GOTO_TARGET
C	Start 360° terrain scan	cmd_scan()	state == HOVER
S	Show path + start flight	cmd_toggle_path()	Path planned via cmd_goto
E	Enter exploration mode	cmd_start_exploration()	state == HOVER
1–5	Fly to Pad 1–5	cmd_goto('Pad N')	Map available
H	Fly to Home pad	cmd_goto('Home')	Map available
K	Kill switch (motor cut)	Sets kill_switch_active	Any flight state
A	Toggle A* ↔ Dijkstra	Toggles planner_algorithm	Any state